Sri Lanka Institute of Information Technology

IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 04

Student ID	IT24101365
Student Name	Kamshiga Ganesan
Campus/ Center name	Northern UNI
Specialization	Software Engineering
Batch No	1

Activity 01

Consider the below program and answer the questions.

a. What is the output you can see in the terminal?

```
  GNU nano 7.2                                act1.c
#include<stdio.h>
#include<unistd.h>
int main()
{
 printf("Process ID is: %d\n",getpid() );
 return 0;
}
```

```
kamshiga@kamshiga-VirtualBox: ~
kamshiga@kamshiga-VirtualBox:~$ nano act1.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o act1 act1.c
kamshiga@kamshiga-VirtualBox:~$ ./act1
Process ID is: 3644
kamshiga@kamshiga-VirtualBox:~$
```

b. What is the system call used in the above program?

- System call- getpid()

c. Briefly describe the purpose of the system call you mentioned in a)?

- The purpose of the system call is returns unique ID of current process

d. Remove the include statement, ”#include¡unistd.h¿” from the program and check the output. What can you see in the output? Describe the reason for your obser vation.

```
kamshiga@kamshiga-VirtualBox:~$ nano act1.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o act1 act1.c
act1.c: In function ‘main’:
act1.c:5:31: warning: implicit declaration of function ‘getpid’ [-Wimplicit-function-dec
    5 |  printf("Process ID is: %d\n",getpid() );
      |                               ^~~~~~
kamshiga@kamshiga-VirtualBox:~$
```

Reason is getpid() is declared in unistd.h

Activity 2

Modify the program in activity 1 and print the parent process ID of the current process. Include the modified program and its output into the submission file.

```
GNU nano 7.2                                act2.c
#include<stdio.h>
#include<unistd.h>
int main()
{
 printf("Process ID is: %d\n",getpid() );
 printf("Process ID of parent process is: %d\n",getppid() );
 return 0;
}
```

```
kamshiga@kamshiga-VirtualBox: ~                    kamshiga@kamshig
amshiga@kamshiga-VirtualBox:~$ nano act2.c
amshiga@kamshiga-VirtualBox:~$ gcc -o act2 act2.c
amshiga@kamshiga-VirtualBox:~$ ./act2
rocess ID is: 3175
rocess ID of parent process is: 2810
amshiga@kamshiga-VirtualBox:~$
```

Activity 3

1.Modify the program in activity 2, and add the system call, sleep( ) after printing the parent process ID. Let the current process to sleep 30 seconds.

```
  GNU nano 7.2                                act3.c
#include<stdio.h>
#include<unistd.h>
int main()
{
 printf("Process ID is: %d\n",getpid() );
 printf("Process ID of parent process is: %d\n",getppid() );

 sleep(30);

 return 0;
}
```

```
kamshiga@kamshiga-VirtualBox: ~    kamshiga@kamshiga-VirtualBox: ~    kam
kamshiga@kamshiga-VirtualBox:~$ nano act3.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o act3 act3.c
kamshiga@kamshiga-VirtualBox:~$ ./act3
Process ID is: 3165
Process ID of parent process is: 2860
kamshiga@kamshiga-VirtualBox:~$ ./act3
Process ID is: 3260
Process ID of parent process is: 2860
kamshiga@kamshiga-VirtualBox:~$
```

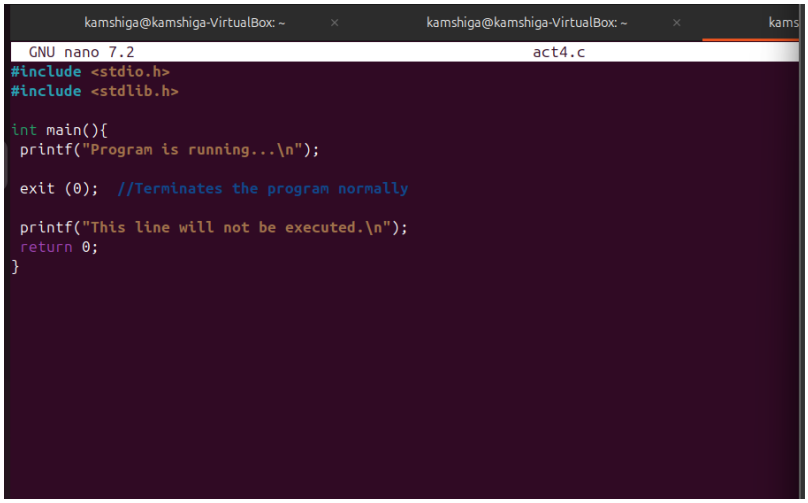
2.Then open a new terminal and type the below UNIX command. pstree-p

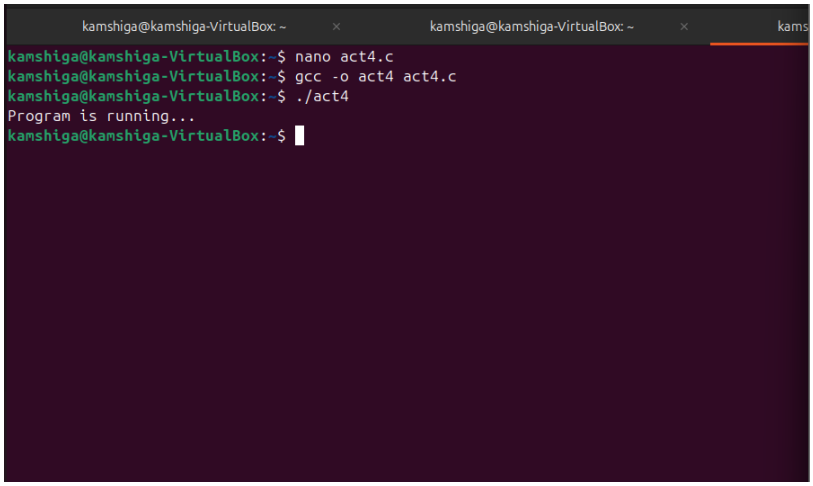
```
kamshiga@kamshiga-VirtualBox: ~    kamshiga@kamshiga-VirtualBox: ~    kams
                                   ├─{gnome-shell}(2061)
                                   ├─{gnome-shell}(2062)
                                   ├─{gnome-shell}(2063)
                                   ├─{gnome-shell}(2598)
                                   └─{gnome-shell}(3153)
                   ├─gnome-shell-cal(2095)─┬─{gnome-shell-cal}(2100)
                                           ├─{gnome-shell-cal}(2101)
                                           ├─{gnome-shell-cal}(2103)
                                           ├─{gnome-shell-cal}(2104)
                                           ├─{gnome-shell-cal}(2105)
                                           └─{gnome-shell-cal}(2374)
                   ├─gnome-terminal-(2848)─┬─bash(2860)───act3(3260)
                                           ├─bash(3213)───pstree(3261)
                                           ├─bash(3247)───nano(3254)
                                           ├─{gnome-terminal-}(2849)
                                           ├─{gnome-terminal-}(2850)
                                           ├─{gnome-terminal-}(2851)
                                           ├─{gnome-terminal-}(2852)
                                           ├─{gnome-terminal-}(2854)
                                           └─{gnome-terminal-}(2859)
```

The sleep(30) function is added so the process stays active for 30 seconds.
During this time, using pstree -p shows the process hierarchy.
It displays the parent-child relationship between processes.

Activity 4

Type the program given below and understand the behavior of exit (0). Explain the reason for not printing the message, ”This line will not be executed”.





Reason: exit (0) terminates the process immediately. Because Code after exit (0) never runs

Activity 5

Type the below program and execute. Observe the output and explain the reason for printing ”Hello world” twice.

```
kamshiga@kamshiga-VirtualBox: ~   kamshiga@kamshiga-VirtualBox: ~   kamshiga@kamshiga-VirtualBox: ~
  GNU nano 7.2                                                    act5.c
#include <stdio.h>
#include <unistd.h>
int main() {
 printf("I am parent\n");
 fork();
 printf("Hello world...!\n");
 return 0;
}
```

```
kamshiga@kamshiga-VirtualBox: ~   kamshiga@kamshiga-VirtualBox: ~   kamshiga@kamshiga-VirtualBox: ~
kamshiga@kamshiga-VirtualBox:~$ nano act5.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o act5 act5.c
kamshiga@kamshiga-VirtualBox:~$ ./act5
I am parent
Hello world...!
Hello world...!
kamshiga@kamshiga-VirtualBox:~$
```

Fork () creates 2 processes (Parent and Child)

So both parent and child execute the same code after fork ().That is why "Hello World" is printed twice.

Activity 6

Modify the program in activity 5, to do the below.

1. Invoke the fork ( ) system call to create a child process.

2. Print the return value of the fork ( ).

3. Separate the parent and the child processes and print their process IDs as shown in figure below.

4. Add the sleep ( ) with an appropriate time in seconds and check the process hier archy.

```
kamshiga@kamshiga-VirtualBox: ~                                  kamshiga@kamsh
  GNU nano 7.2                                                   act6.c
#include <stdio.h>
#include <unistd.h>

int main() {
 pid_t pid;
 printf("Hello World\n");
 pid=fork();
 printf("fork() return value: %d\n",pid);

 if(pid==0){
  printf("I'm the child process\n");
  printf("My PID is: %d\n",getpid());
 }
 else{
  printf("I'm the parent process\n");
  printf("My  PID is: %d\n",getpid());
 }

 sleep(20);
 return 0;
}
```

```
kamshiga@kamshiga-VirtualBox: ~                                  kamshiga@kamshig
kamshiga@kamshiga-VirtualBox:~$ nano act6.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o act6 act6.c
kamshiga@kamshiga-VirtualBox:~$ ./act6
Hello World
fork() return value: 2883
I'm the parent process
My  PID is: 2882
fork() return value: 0
I'm the child process
My PID is: 2883
kamshiga@kamshiga-VirtualBox:~$
```

```
kamshiga@kamshiga-VirtualBox: ~                                  kamshiga@kamshiga-VirtualBox
                                    ├─{gnome-shell}(2054)
                                    ├─{gnome-shell}(2055)
                                    ├─{gnome-shell}(2056)
                                    ├─{gnome-shell}(2057)
                                    ├─{gnome-shell}(2061)
                                    ├─{gnome-shell}(2062)
                                    ├─{gnome-shell}(2063)
                                    ├─{gnome-shell}(2064)
                                    ├─{gnome-shell}(2065)
                                    ├─{gnome-shell}(2066)
                                    ├─{gnome-shell}(2067)
                                    ├─{gnome-shell}(2068)
                                    └─{gnome-shell}(2579)
                        ├─gnome-shell-cal(2146)─┬─{gnome-shell-cal}(2153)
                                                ├─{gnome-shell-cal}(2154)
                                                ├─{gnome-shell-cal}(2156)
                                                ├─{gnome-shell-cal}(2158)
                                                ├─{gnome-shell-cal}(2159)
                                                └─{gnome-shell-cal}(2422)
                        ├─gnome-terminal-(2759)─┬─bash(2772)───act6(2882)───act6(2883)
                                                ├─bash(2875)───pstree(2884)
                                                ├─{gnome-terminal-}(2761)
                                                ├─{gnome-terminal-}(2762)
                                                ├─{gnome-terminal-}(2763)
                                                ├─{gnome-terminal-}(2764)
                                                ├─{gnome-terminal-}(2766)
                                                └─{gnome-terminal-}(2771)
                        ├─goa-daemon(2391)───{goa-daemon}(2419)
```

Activity 7

Type the below program and execute. Observe the output and explain the behavior of execl().

```
  GNU nano 7.2                                act7.c
#include <stdio.h>
#include <unistd.h>
int main()
{
 printf("here comes the date. \n");
 execl("/bin/date","date",NULL);

 printf("that was the date. \n");
 return 0;
}
```

```
kamshiga@kamshiga-VirtualBox:~$ nano act7.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o act7 act7.c
kamshiga@kamshiga-VirtualBox:~$ ./act7
here comes the date.
Sat Feb 21 10:50:53 PM +0530 2026
kamshiga@kamshiga-VirtualBox:~$
```

The execl() system call replaces the current process with a new program.
If execl() executes successfully, the original program stops running.
If it fails, it returns -1 and the original program continues.

Activity 8

Following program demonstrates the sue of system() library function. It is not a system call but, internally uses system calls to execute commands

```
GNU nano 7.2                                act8.c *
#include <stdio.h>
#include <stdlib.h>

int main(){
 printf("here comes the date. \n");
 system("date");
 printf("that was the date");
 return 0;
}
```

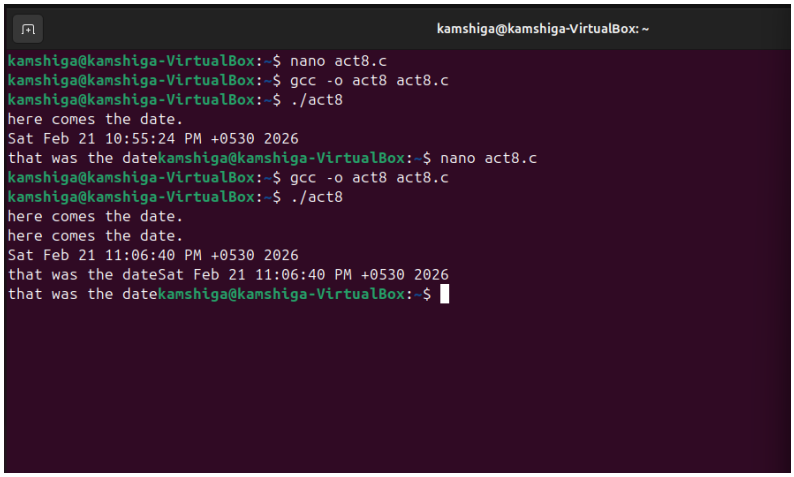
```
kamshiga@kamshiga-VirtualBox: ~
kamshiga@kamshiga-VirtualBox:~$ nano act8.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o act8 act8.c
kamshiga@kamshiga-VirtualBox:~$ ./act8
here comes the date.
Sat Feb 21 10:55:24 PM +0530 2026
that was the datekamshiga@kamshiga-VirtualBox:~$
```

Modify the above program by adding fork () prior to system () function. Observe the output and explain the output.

Adding fork () prior to system () function.

```
GNU nano 7.2                                act8.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(){
 fork();
 printf("here comes the date. \n");
 system("date");
 printf("that was the date");
 return 0;
}
```



The system() function executes a shell command like date.
It internally creates a new process to run the command.
When fork() is added before system(), both parent and child execute it, so the command runs twice.